



UNIVERSITÉ PARIS-SACLAY

Deep Optimal Stopping

Master 2 : Quantitative Finance

Authors : Manuel Griseri, Daniel Badaire, Clément Chivet, Wen Li

Instructor : Karen Bozarian

April 16, 2026

Contents

1	Introduction and motivation	2
2	Mathematical framework	2
2.1	Notation and assumptions	2
2.2	Auxiliary optimal values	3
3	Decomposition into binary decisions	3
3.1	Representation of a stopping time	3
3.2	Main theorem: optimality by induction	3
4	Approximation by neural networks	5
4.1	Network architecture	5
4.2	Universality of neural networks	6
5	Training by backward induction	9
5.1	Optimization objective at each date	9
5.2	Monte Carlo estimation	9
5.3	Full algorithm	10
6	Bounds, point estimators, and confidence intervals	10
6.1	Lower bound	11
6.2	Upper bound	11
6.2.1	Snell envelope and Doob-Meyer decomposition	11
6.2.2	Dual formulation	11
6.2.3	Construction of the upper bound	11
6.2.4	Estimation by continuation paths	12
6.3	Point estimator and confidence intervals	12
7	Benchmarks	13
7.1	Longstaff-Schwartz method	13
7.2	Binomial method	14
8	Numerical results	16
8.1	Bermudan Max-Call Options	16
8.2	Callable Multi Barrier Reverse Convertibles	19
8.3	Fractional Brownian Motion	19
8.4	Hyperparameter Sensitivity	21
9	Conclusion	22
	Appendix: Reminders on probabilistic tools and additional results	22
	References	25

1 Introduction and motivation

The objective of this report is to present clearly, and in a style that allows simple implementation, the *Deep Optimal Stopping* method proposed by Becker, Cheridito, and Jentzen. The goal is to learn the optimal stopping rule directly from Monte Carlo samples, by representing the stopping time as a sequence of binary decisions learned by neural networks trained through backward induction. In finance, many problems amount to finding the *best time* to make an irreversible decision: exercise an American or Bermudan option, repay a loan, liquidate a position. These problems can all be written as an **optimal stopping problem**:

$$V_0 = \sup_{\tau \in \mathcal{T}} \mathbb{E}[g(\tau, X_\tau)] \quad (1)$$

where:

- $X = (X_n)_{n=0}^N$ is a discrete-time Markov process taking values in $\mathbb{R}^d$,
- $g : \{0, \dots, N\} \times \mathbb{R}^d \rightarrow \mathbb{R}$ is the reward function (the payoff at time n in state x), and we assume $g(n, X_n) \in L^1$ for all n (and L^2 to compute confidence intervals),
- $\mathcal{T}$ is the set of all stopping times with respect to the natural filtration $(\mathcal{F}_n)_{n=0}^N$.

Example 1.1 (Bermudan max-call option). We consider d assets following a correlated Black–Scholes model. The holder of a Bermudan max-call option with strike K and maturity T may exercise the option at $N + 1$ dates $t_0 < t_1 < \dots < t_N = T$ and receives

$$g(n, x) = e^{-rt_n} \left(\max_{1 \leq i \leq d} x^i - K \right)^+$$

Finding the price of this option amounts exactly to solving (1).

2 Mathematical framework

2.1 Notation and assumptions

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. We fix:

- $N \in \mathbb{N}^*$ the number of exercise dates, $d \in \mathbb{N}^*$ the dimension,
- $X = (X_n)_{n=0}^N$ a Markov process taking values in $\mathbb{R}^d$,
- $\mathcal{F}_n = \sigma(X_0, X_1, \dots, X_n)$ the natural filtration,
- $\mathcal{T}$ the set of stopping times: $\tau : \Omega \rightarrow \{0, 1, \dots, N\}$ such that $\{\tau = n\} \in \mathcal{F}_n$ for all n .

Table of main notation:

Symbol	Meaning
N	Number of exercise dates
d	Dimension of the state (number of assets)
$X_n \in \mathbb{R}^d$	State of the process at date n
$g(n, x)$	Reward (payoff) at date n in state x
V_n	Optimal value starting from date n
$f_n : \mathbb{R}^d \rightarrow \{0, 1\}$	Optimal stopping decision at date n
$F^{\theta_n} : \mathbb{R}^d \rightarrow (0, 1)$	Neural network (soft decision)
$f^{\theta_n} : \mathbb{R}^d \rightarrow \{0, 1\}$	Hard decision (threshold at 1/2)
τ^Θ	Learned stopping time
$\hat{L}$	Estimated lower bound
$\hat{U}$	Estimated upper bound

Integrability assumptions:

$$\mathbb{E}[|g(n, X_n)|] < \infty \quad \forall n \in \{0, \dots, N\} \quad (2)$$

$$\mathbb{E}[g(n, X_n)^2] < \infty \quad \forall n \in \{0, \dots, N\} \quad (3)$$

Assumption (2) ensures that problem (1) is well defined. Assumption (3) (stronger) will be used to construct confidence intervals (Section 6.3).

2.2 Auxiliary optimal values

Define the optimal values at each date:

$$V_n = \sup_{\tau \in \mathcal{T}_n} \mathbb{E}[g(\tau, X_\tau)] \quad (4)$$

where $\mathcal{T}_n$ is the set of stopping times τ such that $n \leq \tau \leq N$. In particular, V_0 is the global optimal value that we seek to estimate.

Remark 2.1. At terminal date N , the only possible decision is to stop: $\mathcal{T}_N = \{\tau_N \equiv N\}$, so $V_N = \mathbb{E}[g(N, X_N)]$.

3 Decomposition into binary decisions

3.1 Representation of a stopping time

The fundamental idea is that any stopping time can be viewed as a sequence of binary decisions: at each date n , if the process has not yet been stopped, we decide to stop ($f_n(X_n) = 1$) or to continue ($f_n(X_n) = 0$).

Definition 3.1 (Stopping time induced by binary decisions). Given measurable functions $f_n, f_{n+1}, \dots, f_N : \mathbb{R}^d \rightarrow \{0, 1\}$ with $f_N \equiv 1$, we define the stopping time:

$$\tau_n = \sum_{m=n}^N m \cdot f_m(X_m) \prod_{j=n}^{m-1} (1 - f_j(X_j)) \quad (5)$$

Intuition

Let us decode formula (5) step by step:

- The term $f_m(X_m)$ equals 1 if we decide to stop at date m .
- The product $\prod_{j=n}^{m-1} (1 - f_j(X_j))$ equals 1 if and only if we continued at all dates $n, n+1, \dots, m-1$ (i.e., we have not yet been stopped).
- Hence each term in the sum equals m if and only if we stop for the *first time* at date m .
- The condition $f_N \equiv 1$ guarantees that we stop no later than N .

The following result will allow us, in order to find an approximate solution to the optimal stopping problem, to consider stopping times of the form (5).

3.2 Main theorem: optimality by induction

The key result of the paper shows that, in order to optimize over all stopping times, it is enough to find sequentially the best decision at each date.

Theorem 3.2 (Becker *et al.*, Theorem 1). Let $n \in \{0, \dots, N-1\}$ and $\tau_{n+1} \in \mathcal{T}_{n+1}$ be a stopping time of the form (5) induced by decisions $f_{n+1}, \dots, f_N : \mathbb{R}^d \rightarrow \{0, 1\}$ with $f_N \equiv 1$.

Then there exists a measurable function $f_n : \mathbb{R}^d \rightarrow \{0, 1\}$ such that the stopping time τ_n defined by (5) satisfies:

$$\mathbb{E}[g(\tau_n, X_{\tau_n})] \geq V_n - (V_{n+1} - \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}})]) \quad (6)$$

Moreover, this optimal function is given explicitly by:

$$f_n(x) = \begin{cases} 1 & \text{if } g(n, x) \geq h_n(x) \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

where $h_n(x) = \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}}) \mid X_n = x]$ is the optimal continuation value.

Intuition

The theorem says that the optimal rule is simple: **exercise if and only if the immediate gain $g(n, X_n)$ exceeds the continuation value $h_n(X_n)$** . This is exactly the intuitive rule of any rational agent.

The term in parentheses in (6) measures the error made if τ_{n+1} was not exactly optimal (but only close to optimal). If τ_{n+1} is perfectly optimal, $V_{n+1} - \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}})] = 0$ and we recover $\mathbb{E}[g(\tau_n, X_{\tau_n})] \geq V_n$, which is exact optimality.

Proof. Set $\varepsilon = V_{n+1} - \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}})] \geq 0$, and let $T \in \mathcal{T}_n$ be an arbitrary stopping time.

By the Markov property, since $g(\tau_{n+1}, X_{\tau_{n+1}})$ is $\sigma(X_{n+1}, \dots, X_N)$ -measurable, we have

$$\mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}}) \mid \mathcal{F}_n] = \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}}) \mid X_n]$$

which is therefore $\sigma(X_n)$ -measurable: by the Doob-Dynkin lemma 9.1, there exists a measurable function $h_n : \mathbb{R}^d \rightarrow \mathbb{R}$ such that $h_n(X_n) = \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}}) \mid X_n] = \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}}) \mid \mathcal{F}_n]$.

Define the events (all $\mathcal{F}_n$ -measurable):

$$D = \{g(n, X_n) \geq h_n(X_n)\} \quad \text{and} \quad E = \{T = n\}$$

Set $\tau_n = n \cdot \mathbb{1}_D + \tau_{n+1} \cdot \mathbb{1}_{D^c} \in \mathcal{T}_n$, and $\tilde{\tau} = \tau_{n+1} \mathbb{1}_E + T \mathbb{1}_{E^c} \in \mathcal{T}_{n+1}$.

Since $\mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}})] = V_{n+1} - \varepsilon \geq \mathbb{E}[g(\tilde{\tau}, X_{\tilde{\tau}})] - \varepsilon$ because $\tilde{\tau} \in \mathcal{T}_{n+1}$, we have,

$$\begin{aligned} \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}})] &\geq \mathbb{E}[g(\tilde{\tau}, X_{\tilde{\tau}})] - \varepsilon \\ &= \mathbb{E}[g(\tilde{\tau}, X_{\tilde{\tau}})(\mathbb{1}_E + \mathbb{1}_{E^c})] - \varepsilon \\ &\geq \mathbb{E}[g(\tilde{\tau}, X_{\tilde{\tau}})\mathbb{1}_{E^c}] + \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}})\mathbb{1}_E] - \varepsilon \end{aligned}$$

Then, subtracting $\mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}})\mathbb{1}_E]$ from both sides, we obtain:

$$\begin{aligned} \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}})] &\geq \mathbb{E}[g(\tilde{\tau}, X_{\tilde{\tau}})\mathbb{1}_{E^c}] - \varepsilon \\ (\text{definition of } E^c) &= \mathbb{E}[g(T, X_T)\mathbb{1}_{E^c}] - \varepsilon \end{aligned}$$

Similarly, now studying $\tau_n \in \mathcal{T}_n$:

$$\begin{aligned} \mathbb{E}[g(\tau_n, X_{\tau_n})] &= \mathbb{E}[g(\tau_n, X_{\tau_n})(\mathbb{1}_D + \mathbb{1}_{D^c})] \\ (\text{definition of } h_n \mid D^c \in \mathcal{F}_n) &= \mathbb{E}[g(n, X_n)\mathbb{1}_D + h_n(X_n)\mathbb{1}_{D^c}] \\ (\text{definition of } D) &\geq \mathbb{E}[g(n, X_n)\mathbb{1}_E + h_n(X_n)\mathbb{1}_{E^c}] \\ (\text{definition of } h_n \mid E^c \in \mathcal{F}_n) &= \mathbb{E}[g(n, X_n)\mathbb{1}_E + g(\tau_{n+1}, X_{\tau_{n+1}})\mathbb{1}_{E^c}] \\ (\text{previous calculation}) &\geq \mathbb{E}[g(n, X_n)\mathbb{1}_E + g(T, X_T)\mathbb{1}_{E^c}] \\ &\geq \mathbb{E}[g(T, X_T)] - \varepsilon \end{aligned}$$

(To make h_n appear in the previous calculation, we used the fact that $h_n(X_n) = \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}}) \mid \mathcal{F}_n]$, hence for $D^c \in \mathcal{F}_n$, $\mathbb{E}[h_n(X_n) \mathbb{1}_{D^c}] = \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}}) \mathbb{1}_{D^c}]$.)

Since $T \in \mathcal{T}_n$ is arbitrary, $\mathbb{E}[g(\tau_n, X_{\tau_n})] \geq \sup_{T \in \mathcal{T}_n} \mathbb{E}[g(T, X_T)] - \varepsilon = V_n - \varepsilon$.

The function f_n given by (7) indeed satisfies $\mathbb{1}_D = f_n(X_n)$, and $\tau_n = n f_n(X_n) + \tau_{n+1}(1 - f_n(X_n))$ is of the form (5). $\square$

Corollary 3.3 (Globally optimal stopping time). *Applying Theorem 3.2 recursively from $n = N-1$ down to $n = 0$, we obtain an optimal stopping time:*

$$\tau^* = \sum_{n=1}^N n \cdot f_n(X_n) \prod_{j=0}^{n-1} (1 - f_j(X_j)) \quad (8)$$

with f_n given by (7), such that $\mathbb{E}[g(\tau^*, X_{\tau^*})] = V_0$.

Remark 3.4. In most financial applications, $X_0 = x_0$ is deterministic. In this case, f_0 is simply a constant $f_0 \in \{0, 1\}$: we compare $g(0, x_0)$ with the Monte Carlo estimate of the continuation value.

4 Approximation by neural networks

4.1 Network architecture

The continuation value $h_n(x) = \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}}) \mid X_n = x]$ is analytically unknown and difficult to compute. We approximate it by a *deep* neural network.

Since the hard decision $f_n \in \{0, 1\}$ is not differentiable, we introduce a network with **continuous output** (soft probability):

Definition 4.1 (Stopping decision network). For integers $I \geq 2$ (depth) and $q_1, \dots, q_{I-1}$ (hidden-layer widths), we define the network $F^\theta : \mathbb{R}^d \rightarrow (0, 1)$ by:

$$F^\theta = \sigma_{\text{sig}} \circ a_I^\theta \circ \varphi_{q_{I-1}} \circ a_{I-1}^\theta \circ \dots \circ \varphi_{q_1} \circ a_1^\theta \quad (9)$$

where:

- $I, (q_i)_{i=1 \dots I-1}$ are integers giving the dimension and depth of the network,
- $a_i^\theta(x) = A_i x + b_i$ are affine maps $\mathbb{R}^{q_{i-1}} \rightarrow \mathbb{R}^{q_i}$ with $q_0 = d$ and $q_I = 1$. (A_i and b_i are the learnable parameters),
- $\varphi_j(x_1, \dots, x_j) = (x_1^+, \dots, x_j^+)$ is the componentwise ReLU activation function,
- $\sigma_{\text{sig}}(x) = \frac{e^x}{1+e^x} = \frac{1}{1+e^{-x}}$ is the sigmoid (logistic) function, taking values in $(0, 1)$ (since we want to approximate indicator functions).

We therefore seek a parameter θ_n such that f^{θ_n} approximates f_n well. We thus want to determine $\theta_n \in \mathbb{R}^q$ maximizing

$$\mathbb{E} \left[g(n, X_n) F^{\theta_n} + (g_{\tau_{n+1}}, X_{\tau_{n+1}})(1 - F^{\theta_n}) \right] \quad (10)$$

Once such a θ_n has been found, the corresponding **decision** is:

$$f^{\theta_n}(x) = \mathbb{1}_{[0, +\infty)}(F^{\theta_n}(x) - 1/2) = \begin{cases} 1 & \text{if } F^{\theta_n}(x) \geq 1/2 \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

Parameters in the original paper:

- Depth: $I = 3$ (2 hidden layers).
- Width: $q_1 = q_2 = d + 40$.
- Xavier initialization: sets the initial weights to values that keep the scale of activations roughly the same across layers. This helps prevent signals from becoming too large or too small during training, making learning more stable and efficient.
- Batch normalization before each ReLU: normalizes the activations within a mini-batch so that they have a stable mean and variance during training. This stabilizes and speeds up learning, allowing the network to train faster and reducing sensitivity to the choice of initialization and learning rate.

Schematic representation of the network:

$$\begin{array}{c}
 \underbrace{x \in \mathbb{R}^d}_{\text{state } X_n} \xrightarrow{\text{Affine+BN+ReLU}} \mathbb{R}^{q_1} \xrightarrow{\text{Affine+BN+ReLU}} \mathbb{R}^{q_2} \\
 \\
 \xrightarrow{\text{Affine+Sigmoid}} \underbrace{F^{\theta_n}(x) \in (0, 1)}_{\text{stopping prob.}} \xrightarrow{\geq 1/2?} \underbrace{f^{\theta_n}(x) \in \{0, 1\}}_{\text{decision}}
 \end{array}$$

4.2 Universality of neural networks

The following proposition guarantees that approximation by neural networks can be made arbitrarily accurate.

Proposition 4.2 (Universal approximation, Proposition 4 of the paper). *Let $n \in \{0, \dots, N-1\}$ and $\tau_{n+1} \in \mathcal{T}_{n+1}$ be fixed. For any $I \geq 2$ and any $\varepsilon > 0$, there exist integers $q_1, \dots, q_{I-1}$ such that:*

$$\begin{aligned}
 & \sup_{\theta \in \mathbb{R}^q} \mathbb{E} \left[g(n, X_n) f^\theta(X_n) + g(\tau_{n+1}, X_{\tau_{n+1}}) (1 - f^\theta(X_n)) \right] \\
 & \geq \sup_{f \in \mathcal{D}} \mathbb{E} \left[g(n, X_n) f(X_n) + g(\tau_{n+1}, X_{\tau_{n+1}}) (1 - f(X_n)) \right] - \varepsilon
 \end{aligned} \tag{12}$$

where $\mathcal{D}$ is the set of all measurable functions $f : \mathbb{R}^d \rightarrow \{0, 1\}$.

Intuition

This proposition says that a neural network of depth $I \geq 2$, with sufficiently many neurons in the hidden layers, can approximate the optimal stopping rule to any desired precision. This is the density result (universal approximation theorem) adapted to this setting.

Proof. Fix $n \in \{0, \dots, N-1\}$, $\tau_{n+1} \in \mathcal{T}_{n+1}$, $I \geq 2$, and $\varepsilon > 0$.

To simplify notation, let:

$$\begin{aligned}
 \Phi(f) &= \mathbb{E} \left[g(n, X_n) f(X_n) + g(\tau_{n+1}, X_{\tau_{n+1}}) (1 - f(X_n)) \right] \\
 \Phi^* &= \sup_{f \in \mathcal{D}} \Phi(f)
 \end{aligned}$$

We want to show that $\sup_{\theta} \Phi(f^\theta) \geq \Phi^* - \varepsilon$.

Step 1 : Existence of an almost optimal rule

By definition of the supremum, for any $\varepsilon > 0$, there exists $\tilde{f} \in \mathcal{D}$ such that:

$$\Phi(\tilde{f}) \geq \Phi^* - \frac{\varepsilon}{4} \quad (13)$$

We may write $\tilde{f} = \mathbb{1}_A$ for the Borel set $A = \{x \in \mathbb{R}^d : \tilde{f}(x) = 1\}$.

Remark 4.3. We may rewrite Φ to highlight the role of f :

$$\Phi(f) = \mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}})] + \mathbb{E}[(g(n, X_n) - g(\tau_{n+1}, X_{\tau_{n+1}}))f(X_n)]$$

Thus, $f(x) = 1$ is optimal at x if and only if $g(n, x)$ exceeds the continuation value, which is consistent with Theorem 1 of the paper.

Step 2 : Approximation by a rule on a compact set

The following two maps define finite Borel measures on $\mathbb{R}^d$ (by assumption H1):

$$B \mapsto \mathbb{E}[|g(n, X_n)| \mathbb{1}_B(X_n)], \quad B \mapsto \mathbb{E}[|g(\tau_{n+1}, X_{\tau_{n+1}})| \mathbb{1}_B(X_n)]$$

By Lemma 9.3 (tightness of finite measures), each is tight. Therefore there exists a compact set $K \subseteq A$ such that:

$$\Phi(\mathbb{1}_K) \geq \Phi(\mathbb{1}_A) - \frac{\varepsilon}{4} \geq \Phi^* - \frac{\varepsilon}{2} \quad (14)$$

Intuition

We restrict the rule $\tilde{f} = \mathbb{1}_A$ to exercise only on the compact set $K \subseteq A$. The loss in value is at most $\varepsilon/4$ because the probability mass outside K (i.e., in $A \setminus K$) is arbitrarily small. On this compact set, approximation by neural networks becomes possible.

Justification of (14). Let $\delta(x) = g(n, x) - g(\tau_{n+1}, X_{\tau_{n+1}})$. Then $\Phi(\mathbb{1}_A) - \Phi(\mathbb{1}_K) = \mathbb{E}[\delta(X_n) \mathbb{1}_{A \setminus K}(X_n)]$. Since both measures are tight, we choose a compact K such that the residual mass in $A \setminus K$ is small enough so that $|\mathbb{E}[\delta(X_n) \mathbb{1}_{A \setminus K}(X_n)]| \leq \varepsilon/4$.

Step 3 : Approximation of $\mathbb{1}_K$ by a continuous function

Apply Lemma 9.4 to the compact set K from Step 2. The sequence of continuous functions $(k_j)_{j \geq 1}$ defined by (38) satisfies $k_j \rightarrow \mathbb{1}_K - \mathbb{1}_{K^c}$ pointwise.

We have $\{k_j \geq 0\} = \{\rho_K(x) \leq 1/j\}$, and $\mathbb{1}_{\{k_j \geq 0\}} \rightarrow \mathbb{1}_K$ $\mathbb{P}_{X_n}$ -almost surely (because $\mathbb{P}(X_n \in \partial K) = 0$ for non-atomic measures in general).

By the **dominated convergence theorem** (the domination comes from $|g(n, X_n)| + |g(\tau_{n+1}, X_{\tau_{n+1}})| \in L^1$ by assumption H1), we have:

$$\Phi(\mathbb{1}_{\{k_j \geq 0\}}) \xrightarrow{j \rightarrow \infty} \Phi(\mathbb{1}_K)$$

Therefore there exists $j_0 \in \mathbb{N}$ such that:

$$\Phi(\mathbb{1}_{\{k_{j_0} \geq 0\}}) \geq \Phi(\mathbb{1}_K) - \frac{\varepsilon}{4} \geq \Phi^* - \frac{3\varepsilon}{4} \quad (15)$$

Intuition

The rule “exercise if $k_j(x) \geq 0$ ” is continuous in x , unlike $\mathbb{1}_K$, which is discontinuous on the boundary of K . We have replaced the discrete exercise region by a region defined by a continuous criterion, without losing more than $\varepsilon/4$.

Step 4 : Approximation of k_{j_0} by a ReLU network

The function k_{j_0} is continuous on $\mathbb{R}^d$. We apply Theorem 9.5 to it on the compact set:

$$C_R = \{x \in \mathbb{R}^d : \|x\|_2 \leq R\}$$

for R large enough (to be chosen as a function of ε).

Then there exists a function h of the form (39) such that:

$$\sup_{x \in C_R} |k_{j_0}(x) - h(x)| < \eta$$

for an $\eta > 0$ small enough (to be specified in Step 5).

In particular, for $x \in C_R$:

$$k_{j_0}(x) \geq 0 \implies h(x) \geq -\eta, \quad k_{j_0}(x) < 0 \implies h(x) < \eta$$

hence $\mathbb{1}_{\{k_{j_0} \geq 0\}}(x) = \mathbb{1}_{\{h \geq 0\}}(x)$ except possibly on the set $\{|k_{j_0}(x)| < \eta\}$ (a narrow band around the boundary).

We may choose η and R such that:

$$\Phi(\mathbb{1}_{\{h \geq 0\}}) \geq \Phi(\mathbb{1}_{\{k_{j_0} \geq 0\}}) - \frac{\varepsilon}{4} \geq \Phi^* - \varepsilon \quad (16)$$

Justification. The difference $\Phi(\mathbb{1}_{\{k_{j_0} \geq 0\}}) - \Phi(\mathbb{1}_{\{h \geq 0\}})$ is bounded by:

$$\begin{aligned} & \mathbb{E} \left[|g(n, X_n) - g(\tau_{n+1}, X_{\tau_{n+1}})| \cdot \left| \mathbb{1}_{\{k_{j_0}(X_n) \geq 0\}} - \mathbb{1}_{\{h(X_n) \geq 0\}} \right| \right] \\ & \leq \mathbb{E} \left[|g(n, X_n) - g(\tau_{n+1}, X_{\tau_{n+1}})| \cdot \left(\mathbb{1}_{\{|k_{j_0}(X_n)| < \eta\}} + \mathbb{1}_{C_R^c}(X_n) \right) \right] \end{aligned}$$

By tightness of measures (Lemma 9.3), the term with $\mathbb{1}_{C_R^c}$ is $< \varepsilon/8$ for R large enough. The measure of the band $\{|k_{j_0}| < \eta\}$ tends to 0 as $\eta \rightarrow 0$ (because k_{j_0} is continuous and nonzero away from the boundary ∂K), hence the term with $\mathbb{1}_{\{|k_{j_0}| < \eta\}}$ is $< \varepsilon/8$ for η small enough. This yields (16).

Step 5 : The rule $\mathbb{1}_{\{h \geq 0\}}$ is realized by a depth- I network

By Lemma 9.6, the function $\mathbb{1}_{\{h \geq 0\}} = \mathbb{1}_{[0, +\infty)} \circ h$, where h is of the form (39), is exactly realized by a network of the form (11) of depth I with suitable widths $q_1, \dots, q_{I-1}$.

Indeed, $h(x) = \sum_{i=1}^r (v_i^T x + c_i)^+ - \sum_{i=1}^s (w_i^T x + d_i)^+$ is a linear combination of $r + s$ ReLU neurons. We construct the network as follows:

- First hidden layer ($q_1 = r + s$ neurons): compute the pre-activations $(v_i^T x + c_i)_{1 \leq i \leq r}$ and $(w_i^T x + d_i)_{1 \leq i \leq s}$, then apply ReLU.
- Intermediate layers ($2 \leq \ell \leq I - 1$): identity layers (representable with ReLU as $x = (x)^+ - (-x)^+$).
- Output layer: linear combination $\sum \alpha_i y_i$, then sigmoid.

The hard decision $f^\theta(x) = \mathbb{1}_{[0,+\infty)}(F^\theta(x) - 1/2)$ coincides with $\mathbb{1}_{\{h(x) \geq 0\}}$ because σ_{sig} is an increasing bijection from $\mathbb{R}$ to $(0, 1)$, and $F^\theta(x) \geq 1/2$ iff $h(x) \geq 0$.

Conclusion. Combining Steps 1 to 5:

$$\sup_{\theta \in \mathbb{R}^q} \Phi(f^\theta) \geq \Phi(f^{\theta^*}) = \Phi(\mathbb{1}_{\{h \geq 0\}}) \stackrel{(16)}{\geq} \Phi^* - \varepsilon$$

where θ^* is the parameter corresponding to the network realizing $\mathbb{1}_{\{h \geq 0\}}$. This proves Proposition 4.2. □

The following corollary, combining Theorem 3.2 and Proposition 4.2, justifies the full algorithm.

Corollary 4.4 (Existence of an almost optimal policy). *For any $I \geq 2$ and $\varepsilon > 0$, there exist integers $q_1, \dots, q_{I-1}$ and a parameter vector $\Theta = (\theta_0, \theta_1, \dots, \theta_{N-1}) \in \mathbb{R}^{Nq}$ such that the learned stopping time*

$$\tau^\Theta = \sum_{n=1}^N n \cdot f^{\theta_n}(X_n) \prod_{j=0}^{n-1} (1 - f^{\theta_j}(X_j)) \quad (17)$$

satisfies $\mathbb{E}[g(\tau^\Theta, X_{\tau^\Theta})] \geq V_0 - \varepsilon$.

5 Training by backward induction

5.1 Optimization objective at each date

To train the network at date n , we assume that the networks $f^{\theta_{n+1}}, \dots, f^{\theta_N}$ have already been learned (with $f^{\theta_N} \equiv 1$). They define a future stopping time τ_{n+1} .

At date n , with the soft decision F^θ , the expected reward (if we apply F^θ now and τ_{n+1} afterward) is:

$$\mathbb{E}\left[g(n, X_n)F^\theta(X_n) + g(\tau_{n+1}, X_{\tau_{n+1}})(1 - F^\theta(X_n))\right] \quad (18)$$

We want to **maximize** this quantity with respect to θ .

Remark 5.1. The use of $F^\theta \in (0, 1)$ (soft decision) instead of $f^\theta \in \{0, 1\}$ (hard decision) is essential for the differentiability of the objective with respect to θ , which allows the use of gradient ascent.

5.2 Monte Carlo estimation

We can write $\tau_{n+1} = l_{n+1}(X_{n+1}, \dots, X_{N-1})$ (since $X_N \equiv N$). In practice, we have K simulated trajectories $(x_n^k)_{n=0}^N$, $k = 1, \dots, K$, of the process X . For each trajectory k and each date $n < N - 1$, define:

$$l_{n+1}^k = l_{n+1}(x_{n+1}^k, \dots, x_{N-1}^k) \quad (19)$$

i.e. the stopping time of trajectory k starting from $n + 1$, by applying the already trained decisions $f^{\theta_{n+1}}, \dots, f^{\theta_N}$.

The empirical reward along trajectory k is:

$$r_n^k(\theta) = g(n, x_n^k)F^\theta(x_n^k) + g(l_{n+1}^k, x_{l_{n+1}^k}^k)(1 - F^\theta(x_n^k)) \quad (20)$$

By the law of large numbers, the Monte Carlo estimator:

$$\frac{1}{K} \sum_{k=1}^K r_n^k(\theta) \xrightarrow[K \rightarrow \infty]{\mathbb{P}\text{-a.s.}} \mathbb{E} \left[g(n, X_n) F^\theta(X_n) + g(\tau_{n+1}, X_{\tau_{n+1}}) (1 - F^\theta(X_n)) \right] \quad (21)$$

Since $r_n^k(\theta)$ is differentiable in θ almost everywhere, we can apply **stochastic gradient ascent** (mini-batch Adam).

5.3 Full algorithm

Algorithm 1: Backward induction for deep optimal stopping

Input: Simulated trajectories $(x_n^k)_{n=0}^N$, $k = 1, \dots, K$;

Payoff function g ;

Hyperparameters: depth I , width q , number of steps S , batch size B , learning rate η

Output: Trained networks $\{F^{\theta_n}\}_{n=1}^{N-1}$

```

Initialize  $f^{\theta_N} \equiv 1$  ;                                     // always stop at maturity
for  $n = N - 1, N - 2, \dots, 1$  do
    Initialize  $F^{\theta_n}$  (Xavier) ;
    Compute  $l_{n+1}^k$  for all  $k$  with  $f^{\theta_{n+1}}, \dots, f^{\theta_N}$  ;
    for  $s = 1, 2, \dots, S$  do
        Draw a mini-batch  $\mathcal{B} \subset \{1, \dots, K\}$  of size  $B$ ;
        Compute the objective:  $\mathcal{L}(\theta) = \frac{1}{B} \sum_{k \in \mathcal{B}} r_n^k(\theta)$ ;
        Update  $\theta_n \leftarrow \theta_n + \eta \cdot \nabla_{\theta} \mathcal{L}(\theta)$  ;           // Adam
    end
    Freeze the network  $F^{\theta_n}$  ;
end

```

Warning

The same trajectories $(x_n^k)_{n=0}^N$ are used to train all networks $F^{\theta_1}, \dots, F^{\theta_{N-1}}$. This is possible because the Markov property of X guarantees that the trajectories remain valid for all future horizons.

Remark 5.2 (Practical parameters from the paper). The authors use, on GPU (NVIDIA GeForce GTX 1080):

- Training trajectories: $K = 8\,192$ per mini-batch
- Training steps: $S = 3\,000 + d$
- Width: $q_1 = q_2 = d + 40$
- Optimizer: Adam, Xavier initialization, Batch Normalization

For CPU execution (limited resources), one would need to reduce, for example: $q_1 = q_2 = d + 10$, $S = 500$, $B = 512$.

6 Bounds, point estimators, and confidence intervals

After training, the networks provide a candidate stopping time τ^Θ . The optimal value V_0 can be bracketed by a **lower bound** and an **upper bound**.

6.1 Lower bound

Once τ^Θ has been learned, the quantity $L = \mathbb{E}[g(\tau^\Theta, X_{\tau^\Theta})]$ is a lower bound for V_0 by definition of the supremum.

It is estimated on a **new independent set** of K_L trajectories $(y_n^k)_{n=0}^N$, $k = 1, \dots, K_L$, drawn independently from the training trajectories. For each trajectory, let l^k denote the stopping time τ^Θ applied to (y_n^k) . The estimator

$$\hat{L} = \frac{1}{K_L} \sum_{k=1}^{K_L} g(l^k, y_{l^k}^k) \quad (22)$$

is **unbiased** for L , and by the law of large numbers, $\hat{L} \xrightarrow{\mathbb{P}\text{-a.s.}} L$ as $K_L \rightarrow \infty$.

6.2 Upper bound

6.2.1 Snell envelope and Doob-Meyer decomposition

The **Snell envelope** $(H_n)_{n=0}^N$ of the reward process is the smallest supermartingale dominating $(g(n, X_n))_{n=0}^N$:

$$H_n = \operatorname{ess\,sup}_{\tau \in \mathcal{T}_n} \mathbb{E}[g(\tau, X_\tau) \mid \mathcal{F}_n]$$

Its Doob-Meyer decomposition is $H_n = H_0 + M_n^H - A_n^H$, where M^H is an $(\mathcal{F}_n)$ -martingale null at 0 and A^H is a predictable increasing process null at 0.

6.2.2 Dual formulation

Proposition 6.1 (Dual upper bound, Proposition 7 of the paper). *Let $(\varepsilon_n)_{n=0}^N$ be a sequence of integrable random variables with $\mathbb{E}[\varepsilon_n \mid \mathcal{F}_n] = 0$ for all n . For any $(\mathcal{F}_n)$ -martingale $(M_n)_{n=0}^N$ null at 0:*

$$V_0 \leq \mathbb{E} \left[\max_{0 \leq n \leq N} (g(n, X_n) - M_n - \varepsilon_n) \right] \quad (23)$$

Proof. Let τ be any stopping time. Since (M_n) is a martingale, the optional stopping theorem gives $\mathbb{E}[M_\tau] = M_0 = 0$ and $\mathbb{E}[\varepsilon_\tau] = \sum_{n=0}^N \mathbb{E}[\mathbb{1}_{\{\tau=n\}} \varepsilon_n] = 0$. Therefore:

$$\begin{aligned} \mathbb{E}[g(\tau, X_\tau)] &= \mathbb{E}[g(\tau, X_\tau) - M_\tau - \varepsilon_\tau] \\ &\leq \mathbb{E} \left[\max_{0 \leq n \leq N} (g(n, X_n) - M_n - \varepsilon_n) \right] \end{aligned}$$

Taking the supremum over τ , we obtain (23). □

Intuition

The closer (M_n) is to the martingale part M^H of the Snell envelope, the tighter the upper bound. The method therefore consists in constructing a martingale M close to M^H from the learned stopping time τ^Θ .

6.2.3 Construction of the upper bound

We approximate H_n by:

$$H_n^\Theta = \mathbb{E}[g(\tau_n^\Theta, X_{\tau_n^\Theta}) \mid \mathcal{F}_n]$$

where τ_n^Θ is the learned stopping time starting from n . The martingale part M^Θ of (H_n^Θ) has increments:

$$\Delta M_n^\Theta = M_n^\Theta - M_{n-1}^\Theta = f^{\theta_n}(X_n) g(n, X_n) + (1 - f^{\theta_n}(X_n)) C_n^\Theta - C_{n-1}^\Theta \quad (24)$$

where $C_n^\Theta = \mathbb{E}[g(\tau_{n+1}^\Theta, X_{\tau_{n+1}^\Theta}) \mid X_n]$ is the **continuation value** at n .

6.2.4 Estimation by continuation paths

To estimate C_n^Θ at a point z_n^k , we simulate J **independent continuation paths** $\tilde{z}_{n+1}^{k,j}, \dots, \tilde{z}_N^{k,j}$, $j = 1, \dots, J$, starting from z_n^k :

$$C_n^k = \frac{1}{J} \sum_{j=1}^J g\left(\tau_{n+1}^\Theta, \tilde{z}_{\tau_{n+1}^\Theta}^{k,j}\right) \quad (25)$$

The noisy increments are:

$$\Delta M_n^k = f^{\theta_n}(z_n^k) g(n, z_n^k) + (1 - f^{\theta_n}(z_n^k)) C_n^k - C_{n-1}^k \quad (26)$$

and the estimated martingale is $M_n^k = \sum_{m=1}^n \Delta M_m^k$.

The estimator of the upper bound is then:

$$\hat{U} = \frac{1}{K_U} \sum_{k=1}^{K_U} \max_{0 \leq n \leq N} (g(n, z_n^k) - M_n^k) \quad (27)$$

6.3 Point estimator and confidence intervals

Point estimator:

$$\hat{V}_0 = \frac{\hat{L} + \hat{U}}{2} \quad (28)$$

Asymptotic 95% confidence intervals:

By the central limit theorem (under assumption (3)), $\hat{L}$ is approximately $\mathcal{N}(L, \hat{\sigma}_L^2/K_L)$ and $\hat{U}$ is approximately $\mathcal{N}(U, \hat{\sigma}_U^2/K_U)$, with:

$$\hat{\sigma}_L^2 = \frac{1}{K_L - 1} \sum_{k=1}^{K_L} (g(l^k, y_{l^k}^k) - \hat{L})^2, \quad \hat{\sigma}_U^2 = \frac{1}{K_U - 1} \sum_{k=1}^{K_U} \left(\max_{0 \leq n \leq N} (g(n, z_n^k) - M_n^k) - \hat{U} \right)^2$$

An asymptotic confidence interval at level $1 - \alpha$ for V_0 is:

$$\left[\hat{L} - z_{\alpha/2} \frac{\hat{\sigma}_L}{\sqrt{K_L}}, \hat{U} + z_{\alpha/2} \frac{\hat{\sigma}_U}{\sqrt{K_U}} \right] \quad (29)$$

where $z_{\alpha/2}$ is the $1 - \alpha/2$ quantile of the standard normal law ($z_{0.025} = 1.96$ for a 95% CI).

Remark 6.2. The confidence interval $[\hat{L}, \hat{U}]$ (without the statistical uncertainty terms) is a **certified interval**: $V_0 \in [\hat{L}, \hat{U}]$ almost surely for budgets $K_L, K_U \rightarrow \infty$. The width of the window $\hat{U} - \hat{L}$ measures the accuracy of the learned policy.

7 Benchmarks

This section presents two classical benchmarks for optimal stopping problems. They will be used to compare the results obtained by the deep optimal stopping method in the numerical example of the Bermudan max-call option. The first one is the Longstaff-Schwartz regression-based Monte Carlo method, and the second one is the binomial tree method. Both provide useful reference values, although their practical efficiency strongly depends on the dimension of the problem.

7.1 Longstaff-Schwartz method

The Longstaff-Schwartz method is a simulation-based backward procedure originally designed for American-style derivatives. Its main idea is to estimate, at each exercise date, the *continuation value* by regression on a family of basis functions.

Let us denote by

$$C_n(x) = \mathbb{E}[V_{n+1} \mid X_n = x]$$

the continuation value at date n . The optimal stopping rule is:

$$\tau^* \text{ stops at date } n \iff g(n, X_n) \geq C_n(X_n) \quad (30)$$

Since C_n is not known analytically, the Longstaff-Schwartz method approximates it by a regression of the discounted future cashflows on basis functions of the state. More precisely, for a family of basis functions $\{\psi_1, \dots, \psi_p\}$, one seeks coefficients $(\beta_{n,1}, \dots, \beta_{n,p})$ such that

$$C_n(x) \approx \sum_{j=1}^p \beta_{n,j} \psi_j(x) \quad (31)$$

Given Monte Carlo trajectories, one starts from maturity, where the value is known:

$$V_N = g(N, X_N),$$

and then moves backward. At each date n , one considers the paths that are in the money and regresses the realized continuation cashflows on the basis functions. The stopping rule becomes:

$$\hat{f}_n(x) = \begin{cases} 1 & \text{if } g(n, x) \geq \hat{C}_n(x) \\ 0 & \text{otherwise} \end{cases} \quad (32)$$

Hence the estimated value along one path is obtained recursively by

$$\hat{V}_n = \max(g(n, X_n), \hat{C}_n(X_n)) \quad (33)$$

Advantages.

- It is simple to implement and remains one of the standard benchmarks for Bermudan and American option pricing.
- It is Monte Carlo based, so it adapts naturally to general stochastic dynamics.
- In moderate dimension, it is often fast and accurate enough to provide a strong baseline.

Limitations.

- Its performance depends heavily on the choice of basis functions.

- In high dimension, regression becomes unstable and the required basis family may become very large.

Comparison with the deep method.

- Compared with deep optimal stopping, Longstaff-Schwartz is usually easier to calibrate and more transparent.
- However, the deep method is more flexible because the continuation region is represented by neural networks rather than by a small set of handcrafted basis functions.
- For high-dimensional problems such as the max Bermudan option with many assets, the deep method is generally expected to scale better.

Algorithm 2: Longstaff-Schwartz benchmark

Input: Simulated trajectories $(x_n^k)_{n=0}^N$, $k = 1, \dots, K$; payoff function g ; basis functions $\psi_1, \dots, \psi_p$

Output: Estimated price $\hat{V}_0$ and stopping policy $\{\hat{f}_n\}_{n=0}^{N-1}$

Set the cashflow at maturity:

$$Y_N^k = g(N, x_N^k), \quad k = 1, \dots, K$$

for $n = N - 1, N - 2, \dots, 0$ **do**

 Select the in-the-money paths:

$$\mathcal{I}_n = \{k : g(n, x_n^k) > 0\}$$

 For $k \in \mathcal{I}_n$, define the discounted future cashflow Z_n^k obtained from the current path if one continues;

 Regress $(Z_n^k)_{k \in \mathcal{I}_n}$ on the basis functions $(\psi_j(x_n^k))_{1 \leq j \leq p}$;

 Obtain the fitted continuation value:

$$\hat{C}_n(x) = \sum_{j=1}^p \hat{\beta}_{n,j} \psi_j(x)$$

for $k = 1, \dots, K$ **do**

if $g(n, x_n^k) \geq \hat{C}_n(x_n^k)$ **then**

 stop at date n on path k and set the path cashflow accordingly;

else

 continue and keep the future discounted cashflow;

 Define the stopping rule:

$$\hat{f}_n(x) = \mathbb{1}_{\{g(n,x) \geq \hat{C}_n(x)\}}$$

Estimate the option price by averaging the resulting discounted cashflows over all paths;

7.2 Binomial method

The binomial method is a tree-based discretization of the underlying dynamics. In the one-dimensional Black-Scholes setting, over a time step Δt , the asset price is approximated by

$$S_{n+1} = \begin{cases} uS_n & \text{with probability } q \\ dS_n & \text{with probability } 1 - q \end{cases} \quad (34)$$

where typically

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = e^{-\sigma\sqrt{\Delta t}}, \quad q = \frac{e^{r\Delta t} - d}{u - d} \quad (35)$$

Once the recombining tree is built, one computes the option value by backward induction. At maturity,

$$V_N(S_N) = g(N, S_N)$$

and at each earlier date,

$$V_n(S_n) = \max(g(n, S_n), e^{-r\Delta t}(qV_{n+1}(uS_n) + (1 - q)V_{n+1}(dS_n))) \quad (36)$$

This recursion is the discrete analogue of the Snell envelope recursion: at each node, one compares the immediate exercise payoff with the expected discounted continuation value.

For a Bermudan option, the maximum is taken only at the admissible exercise dates. At the other dates, the value is simply the discounted conditional expectation:

$$V_n(S_n) = e^{-r\Delta t}(qV_{n+1}(uS_n) + (1 - q)V_{n+1}(dS_n)) \quad (37)$$

whenever exercise is not allowed.

Algorithm 3: Binomial tree benchmark

Input: Initial asset price S_0 ; maturity T ; number of time steps N ; interest rate r ; volatility σ ; payoff function g

Output: Estimated price V_0

Set $\Delta t = T/N$;

Compute the tree parameters:

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = e^{-\sigma\sqrt{\Delta t}}, \quad q = \frac{e^{r\Delta t} - d}{u - d}$$

Build the recombining stock-price tree:

$$S_n^{(j)} = S_0 u^j d^{n-j}, \quad j = 0, \dots, n$$

Initialize terminal values:

$$V_N^{(j)} = g(N, S_N^{(j)}), \quad j = 0, \dots, N$$

for $n = N - 1, N - 2, \dots, 0$ **do**

for $j = 0, \dots, n$ **do**

 Compute the continuation value:

$$\text{Cont}_n^{(j)} = e^{-r\Delta t}(qV_{n+1}^{(j+1)} + (1 - q)V_{n+1}^{(j)})$$

if *date n is an exercise date* **then**

 set

$$V_n^{(j)} = \max(g(n, S_n^{(j)}), \text{Cont}_n^{(j)})$$

else

 set

$$V_n^{(j)} = \text{Cont}_n^{(j)}$$

Return $V_0^{(0)}$;

Advantages.

- It is conceptually simple and directly implements dynamic programming.
- In low dimension, it is very accurate and often serves as a reference method.
- It does not require regression, optimization, or training.

Limitations.

- It suffers from the curse of dimensionality: for several assets, the tree grows exponentially and quickly becomes infeasible.
- It is much less flexible than simulation-based methods when the model dynamics are complex.

Comparison with the deep method.

- In one dimension, the binomial method may provide a very strong benchmark and can even be more accurate than a learned method.
- In higher dimension, however, the deep method is much more practical because it works directly with simulated paths in $\mathbb{R}^d$.

8 Numerical results

8.1 Bermudan Max-Call Options

We first consider the pricing of Bermudan max-call options, which is one of the standard benchmark problems in the optimal stopping literature. Under a multi-dimensional Black-Scholes model, the asset prices satisfy

$$S_t^i = S_0^i \exp \left((r - \delta_i - \frac{1}{2}\sigma_i^2)t + \sigma_i W_t^i \right), \quad i = 1, \dots, d$$

At each admissible exercise date, the payoff is given by

$$g(n, x) = e^{-rt_n} \left(\max_{1 \leq i \leq d} x_i - K \right)^+$$

so that the pricing problem can be formulated as the optimal stopping problem

$$\sup_{\tau} \mathbb{E} \left[e^{-r\tau} \left(\max_{1 \leq i \leq d} S_{\tau}^i - K \right)^+ \right] = \sup_{\tau} \mathbb{E} g(\tau, X_{\tau})$$

Why this example matters. This example is particularly useful because it is both classical and challenging. It is classical in the sense that it is widely used as a benchmark in the American/Bermudan option pricing literature, which makes comparison with existing methods meaningful. At the same time, it becomes increasingly difficult in higher dimensions, since traditional lattice-based approaches suffer from the curse of dimensionality. As a result, it provides a natural setting for assessing whether the DOS framework can remain accurate and stable when the state dimension grows.

Pricing results. The numerical results are reported in Table 1. For each configuration, we report the primal estimate (lower policy), the dual estimate (upper dual), the resulting point estimate, and benchmark prices from the Longstaff-Schwartz method (LSM) and the binomial method when available.

Several features emerge from Table 1. First, in low-dimensional settings ($d = 2, 3$), the DOS point estimates are reasonably close to both LSM and binomial benchmarks, indicating that the learned stopping policies are financially meaningful. Second, the DOS method provides an interval between a lower and an upper estimate, whereas LSM only yields a single pricing estimate. This primal-dual gap gives additional information on the quality of the learned stopping policy. Third, within the tested range $d = 2, 3, 5$, the method remains numerically stable and continues to produce consistent estimates as the dimension increases.

Table 1: Pricing results for the Bermudan max-call option

d	S_0	Lower policy	Upper dual	Point estimate	LSM	Binomial
2	90	6.582	8.530	7.556	7.236	8.116
2	100	11.629	14.621	13.125	13.563	14.271
2	110	19.321	21.858	20.590	21.175	21.465
3	90	9.563	11.786	10.675	11.134	11.311
3	100	16.354	19.351	17.853	18.462	19.079
3	110	24.052	28.594	26.323	27.188	27.678
5	90	14.559	17.243	15.901	14.274	—
5	100	23.019	27.058	25.038	23.473	—
5	110	34.362	37.398	35.880	35.253	—

Dual gap and nested simulation budget. To better understand the behavior of the dual estimator, we study the effect of the nested sample size J . The corresponding results are displayed in Figure 1. As J increases, the dual gap decreases monotonically. This is consistent with the theory developed in Section 6: a larger nested sample size improves the approximation of continuation values and therefore yields a tighter dual upper bound.

At the same time, increasing J comes at a computational cost. Hence, Figure 1 illustrates the expected trade-off between bound tightness and computational effort. This is an important practical point, because it shows that the quality of the dual bound can be improved systematically, but only by allocating more resources to nested simulation.

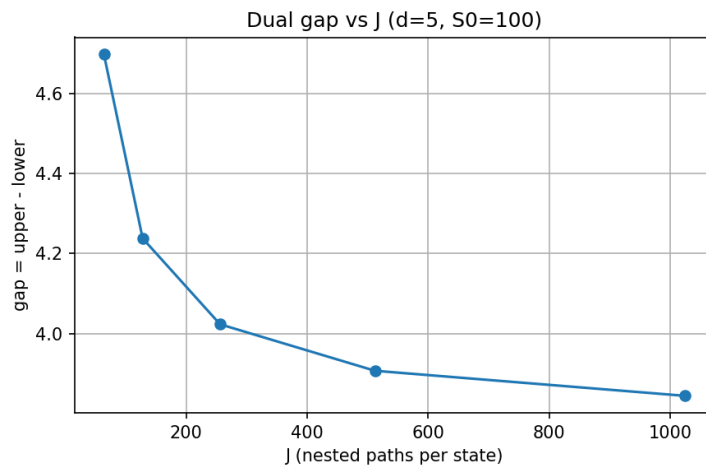


Figure 1: Dual gap (upper – lower) as a function of J

Detailed numerical values are reported in Appendix A.8.

Learned exercise policy. Figure 2 compares the exercise regions obtained from the DOS policy and from the LSM benchmark in the two-dimensional case at $t = 6$. The DOS exercise region exhibits a smooth and coherent boundary, whereas the LSM region appears more fragmented, with several disconnected stopping areas.

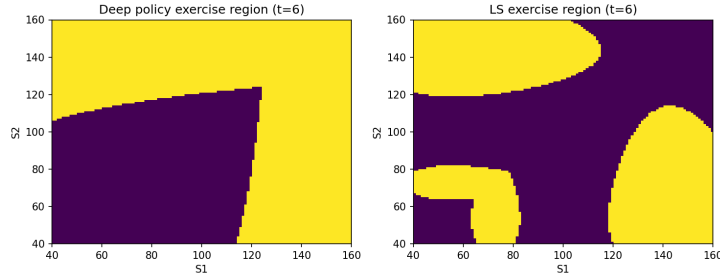


Figure 2: Exercise region in the two-dimensional case at $t = 6$

This difference is informative. Since DOS learns the stopping rule directly, it is able to produce a structured decision boundary in the state space. By contrast, LSM relies on regression of continuation values using basis functions, which may introduce irregularities in the implied stopping rule. Therefore, Figure 2 provides qualitative evidence that the DOS framework captures the exercise policy in a more stable and interpretable way.

Training behavior. To complement the pricing and policy results, Figure 3 reports the training reward over iterations for a representative configuration. We observe a rapid increase in the reward during the first iterations, followed by a relatively stable regime with moderate oscillations. This suggests that the optimization procedure is numerically workable in practice and that the network quickly identifies a high-reward region before stabilizing around it.

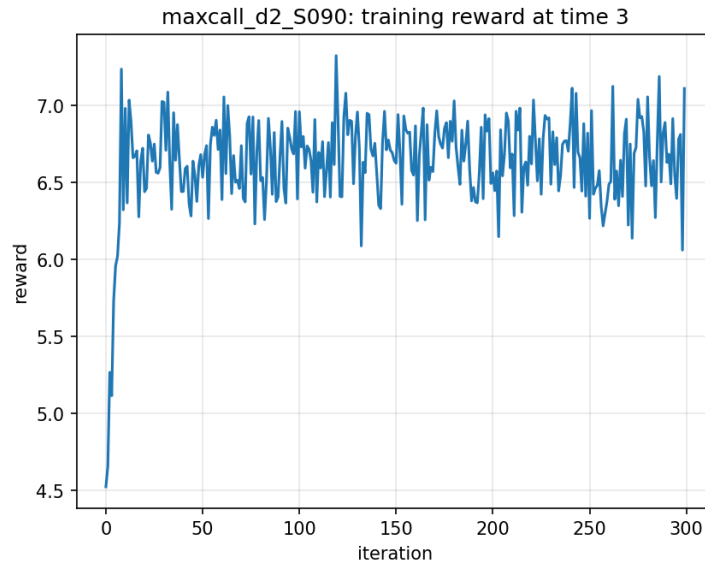


Figure 3: Training reward over iterations

Summary. Overall, the Bermudan max-call example shows that the DOS framework combines three desirable features: competitive pricing performance, interpretable stopping policies, and informative primal-dual bounds. This makes it a strong alternative to classical regression-based methods, especially when one wants not only a price estimate, but also an explicit measure of the quality of the learned stopping rule.

8.2 Callable Multi Barrier Reverse Convertibles

We next consider callable multi barrier reverse convertibles (CMBRC), which involve more complex payoff structures. The payoff depends on barrier events and the worst-performing asset.

The problem can be formulated as

$$\inf_{\tau} \mathbb{E} g(\tau, X_{\tau})$$

where the reward function incorporates coupon payments, early redemption, and terminal payoff conditions depending on barrier events.

Why this example matters. Unlike Bermudan options, the payoff here is highly structured and path-dependent, making it a more challenging test of the flexibility of the DOS framework.

Results. The results are reported in Table 2. We observe that the lower and upper estimates remain very close across different dimensions and correlation levels, indicating that the learned stopping policies are stable.

The callable feature significantly reduces the value compared to the non-callable case, which is consistent with financial intuition.

Interpretation. The closeness of the primal and dual estimates suggests that the neural network is able to capture complex stopping rules even when the payoff structure is significantly more involved than standard benchmark problems.

Small inconsistencies between lower and upper estimates may occur due to Monte Carlo sampling noise, and should be interpreted as numerical estimation effects rather than violations of theoretical bounds.

This example highlights the ability of the DOS framework to generalize beyond simple payoff structures.

Table 2: Pricing results for the callable multi barrier reverse convertible (CMBRC)

d	ρ	Lower bound	Upper bound	Point estimate
2	0.6	100.341	100.355	100.348
2	0.1	99.245	99.302	99.274
3	0.6	98.563	98.607	98.585
3	0.1	96.230	96.261	96.245
5	0.6	96.044	96.052	96.048
5	0.1	91.272	91.293	91.282

8.3 Fractional Brownian Motion

Finally, we consider the problem of optimally stopping a fractional Brownian motion (fBM) with Hurst parameter H . The covariance structure is given by

$$\mathbb{E}[W_t^H W_s^H] = \frac{1}{2} (t^{2H} + s^{2H} - |t - s|^{2H})$$

The optimal stopping problem is

$$\sup_{0 \leq \tau \leq 1} \mathbb{E}[W_{\tau}^H]$$

Since fractional Brownian motion is not Markovian, we embed it into a high-dimensional Markov process by augmenting the state with past values:

$$X_n = (W_{t_n}^H, W_{t_{n-1}}^H, \dots, W_{t_1}^H)$$

Why this example matters. This example is particularly challenging because the underlying process is not Markovian in continuous time. Therefore, it tests whether the DOS framework can handle problems with long-range dependence.

Results. The results are shown in Table 3 and Figure 4. We observe that the value depends strongly on the Hurst parameter H , with higher values corresponding to stronger positive dependence.

For $H = 1/2$, the process reduces to standard Brownian motion, and the optimal value is close to zero, as expected from the martingale property.

Table 3: Pricing results for the fractional Brownian motion example

H	Lower bound	Upper bound	Point estimate
0.01	0.659	1.508	1.083
0.10	0.447	1.050	0.749
0.20	0.279	0.664	0.471
0.30	0.170	0.385	0.277
0.40	0.079	0.184	0.131
0.50	0.005	0.098	0.051
0.60	0.071	0.140	0.105
0.70	0.138	0.220	0.179
0.80	0.203	0.288	0.245
0.90	0.262	0.347	0.305
0.99	0.320	0.387	0.354

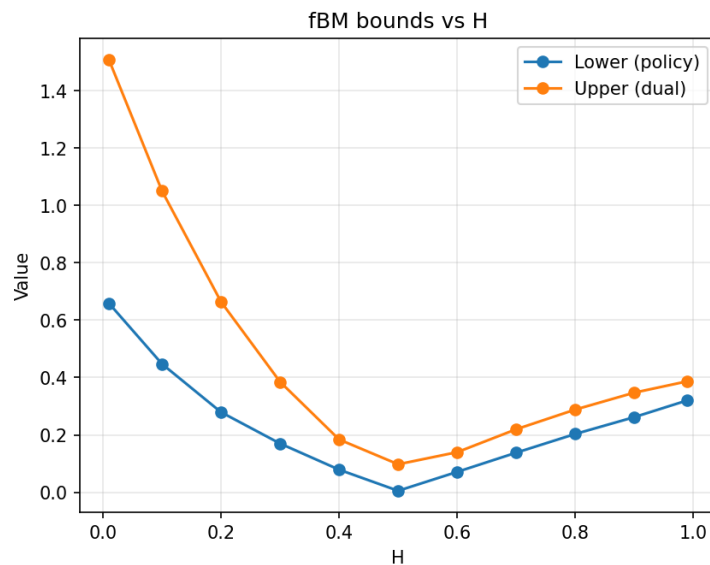


Figure 4: Lower and upper bounds as a function of H

Interpretation. The method remains stable across different values of H , and the dual gap stays relatively small. This suggests that the DOS framework can handle non-Markovian structures when an appropriate state augmentation is used.

Overall, this example demonstrates the robustness of the method to changes in the underlying stochastic dynamics.

8.4 Hyperparameter Sensitivity

We finally study the sensitivity of the method with respect to key hyperparameters. Figure 5 illustrates the effect of network width on the dual gap.

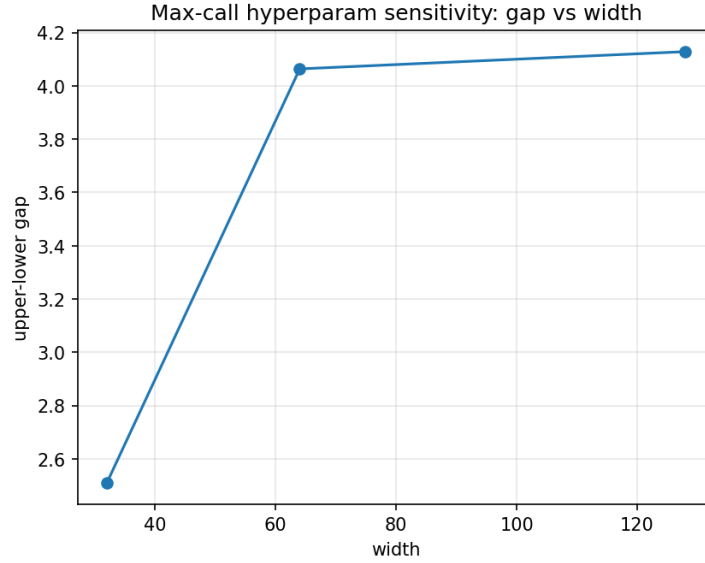
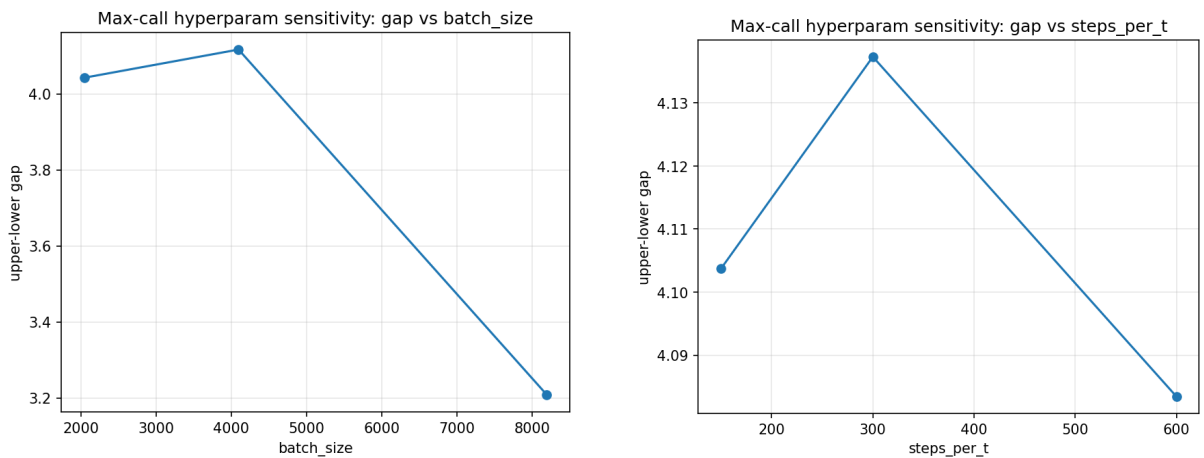


Figure 5: Effect of network width on the dual gap

We observe that different hyperparameters have distinct effects on the dual gap.

Increasing the network width tends to increase the dual gap, which may indicate overfitting or instability when the model becomes excessively flexible.



(a) Effect of batch size on the dual gap

(b) Effect of training steps per time step on the dual gap

Figure 6: Hyperparameter sensitivity of the dual gap for the Bermudan max-call example

Batch size also has a noticeable impact: larger batch sizes tend to reduce the dual gap, suggesting improved stability of the stochastic optimization.

By contrast, the number of training steps per time step has a relatively limited effect within the tested range, indicating that the method is not overly sensitive to this parameter once a reasonable training budget is reached.

Overall, these results highlight the importance of balancing model capacity and training stability when applying deep learning methods to optimal stopping problems.

9 Conclusion

In this report, we implemented and evaluated the Deep Optimal Stopping framework for solving high-dimensional optimal stopping problems. By representing the stopping time as a sequence of binary decisions and approximating these decisions with neural networks, the method transforms a classical dynamic programming problem into a sequence of optimization problems that can be handled using stochastic gradient methods.

The Bermudan max-call experiments demonstrate that the DOS framework achieves competitive pricing performance compared to the Longstaff-Schwartz method and binomial benchmarks in low-dimensional settings. At the same time, the method provides both a lower and an upper estimate, which together form a primal-dual interval. This interval offers an explicit measure of the quality of the learned stopping policy, which is not available in regression-based approaches such as LSM.

Beyond pricing accuracy, the numerical results also provide insight into the behavior of the method. The analysis of the nested simulation parameter J shows that the dual gap can be systematically reduced by increasing the computational budget, illustrating the trade-off between accuracy and efficiency. The visualization of exercise regions reveals that the DOS policy produces smoother and more structured stopping boundaries than LSM, suggesting improved stability and interpretability of the learned decision rule. In addition, the training reward curves indicate that the optimization procedure is numerically stable and converges reliably in practice.

Taken together, these findings highlight the main strength of the DOS framework: it not only produces accurate price estimates, but also learns interpretable stopping policies and provides quantitative error bounds. This makes it particularly attractive in high-dimensional settings where classical methods become difficult to design and scale.

There are, however, several limitations. The construction of the dual upper bound relies on nested Monte Carlo simulation, which can be computationally expensive. Moreover, the performance of the method depends on hyperparameter choices, such as network width, which may affect both stability and the tightness of the dual gap.

Overall, the Deep Optimal Stopping framework offers a flexible and effective alternative to traditional approaches for optimal stopping problems. Future work may focus on improving the efficiency of the dual estimator, developing more robust training strategies, and extending the framework to more general stochastic control problems.

Appendix: Reminders on probabilistic tools and additional results

A.1 Discrete-time Markov process

A process $(X_n)_{n=0}^N$ is **Markovian** if for every Borel set B :

$$\mathbb{P}(X_{n+1} \in B \mid X_0, \dots, X_n) = \mathbb{P}(X_{n+1} \in B \mid X_n) \quad \mathbb{P}\text{-a.s.}$$

Intuition: the future depends on the past only through the present state.

A.2 Snell envelope

The **Snell envelope** $(H_n)_{n=0}^N$ of a process $(Z_n)_{n=0}^N$ is the smallest supermartingale dominating (Z_n) . It satisfies the recursion:

$$H_N = Z_N, \quad H_n = \max(Z_n, \mathbb{E}[H_{n+1} \mid \mathcal{F}_n])$$

The **optimal stopping time** is $\tau^* = \min\{n : H_n = Z_n\}$ (first time when stopping is optimal).

A.3 Doob-Meyer decomposition

Any square-integrable supermartingale (H_n) decomposes uniquely (a.s.) as:

$$H_n = H_0 + M_n - A_n$$

where (M_n) is a martingale null at 0 and (A_n) is an increasing, predictable process null at 0.

A.4 Doob-Dynkin lemma

Lemma 9.1 (Doob-Dynkin). *Let $(\Omega', \mathcal{A}')$ be a measurable space, $T : \Omega \rightarrow \Omega'$ an application. A function $f : \Omega \rightarrow [0, 1]$ (or any interval of $\mathbb{R}$) is $\sigma(T) - \mathcal{B}([0, 1])$ measurable if and only if there exists a measurable function $g : (\Omega', \mathcal{A}') \rightarrow ([0, 1], \mathcal{B}([0, 1]))$ such that $f = g \circ T$.*

If (X_n) is Markovian, then any random variable Z measurable with respect to $\mathcal{F}_n = \sigma(X_0, \dots, X_n)$ can be written as $Z = h(X_n)$ for some measurable function $h : \mathbb{R}^d \rightarrow \mathbb{R}$. In particular: $\mathbb{E}[g(\tau_{n+1}, X_{\tau_{n+1}}) \mid \mathcal{F}_n] = h_n(X_n)$ for a function h_n depending only on X_n .

A.5 Tightness of finite Borel measures

Definition 9.2 (Tight measure). A Borel measure μ on $\mathbb{R}^d$ is said to be **tight** if for every $\varepsilon > 0$, there exists a compact set $K \subset \mathbb{R}^d$ such that $\mu(K^c) < \varepsilon$.

Lemma 9.3 (Tightness of finite Borel measures on $\mathbb{R}^d$). *Every finite Borel measure on $\mathbb{R}^d$ is tight.*

Proof. Let μ be a finite Borel measure on $\mathbb{R}^d$. Set $B_k = \{x \in \mathbb{R}^d : \|x\|_2 \leq k\}$. Then $(B_k)_{k \geq 1}$ is an increasing sequence of compact sets whose union is $\mathbb{R}^d$. By monotone continuity of μ :

$$\mu(B_k) \nearrow \mu(\mathbb{R}^d) < \infty$$

Hence for every $\varepsilon > 0$, there exists k_0 such that $\mu(B_{k_0}^c) < \varepsilon$. The compact set $K = B_{k_0}$ works. $\square$

A.6 Approximation of indicators by continuous functions

Lemma 9.4 (L_μ^1 approximation of compact indicators). *Let μ be a finite Borel measure on $\mathbb{R}^d$ and $K \subset \mathbb{R}^d$ a compact set. Define the distance function to K by:*

$$\rho_K(x) = \inf_{y \in K} \|x - y\|_2$$

and the sequence of continuous functions:

$$k_j(x) = \max\{1 - j \rho_K(x), -1\}, \quad j \in \mathbb{N} \tag{38}$$

Then:

- (i) k_j is continuous, taking values in $[-1, 1]$.
- (ii) $k_j \rightarrow \mathbb{1}_K - \mathbb{1}_{K^c}$ pointwise on $\mathbb{R}^d \setminus \partial K$ as $j \rightarrow \infty$.
- (iii) For every $h \in L^1(\mu)$, $\mathbb{E}_\mu[h \cdot \mathbb{1}_{\{k_j \geq 0\}}] \rightarrow \mathbb{E}_\mu[h \cdot \mathbb{1}_K]$ as $j \rightarrow \infty$.

Proof. (i) The function ρ_K is 1-Lipschitz (triangle inequality), hence continuous. Since $\max\{\cdot, -1\}$ is continuous, so is k_j .

(ii) If $x \in K^\circ$ (interior), then $\rho_K(x) = 0$ and $k_j(x) = 1$ for all j , hence $k_j(x) \rightarrow 1 = \mathbb{1}_K(x) - \mathbb{1}_{K^c}(x)$. If $x \notin K$ (and $x \notin \partial K$), then $\rho_K(x) = \delta > 0$ and $k_j(x) = \max\{1 - j\delta, -1\} = -1$ for j large enough, hence $k_j(x) \rightarrow -1 = \mathbb{1}_K(x) - \mathbb{1}_{K^c}(x)$.

(iii) We have $\{k_j \geq 0\} = \{x : \rho_K(x) \leq 1/j\}$, which is a decreasing sequence of sets converging to K (up to a μ -negligible set, since $\mu(\partial K) = 0$ for almost every measure). By the dominated convergence theorem (domination: $|h| \in L^1(\mu)$), we obtain (iii). $\square$

A.7 Universal approximation by ReLU networks

Theorem 9.5 (Uniform approximation on compact sets by ReLU networks). *Let $h : \mathbb{R}^d \rightarrow \mathbb{R}$ be a continuous function. For every compact set $C \subset \mathbb{R}^d$ and every $\varepsilon > 0$, there exist integers $r, s \geq 1$, vectors $v_1, \dots, v_r, w_1, \dots, w_s \in \mathbb{R}^d$ and scalars $c_1, \dots, c_r, d_1, \dots, d_s \in \mathbb{R}$ such that the function*

$$\hat{h}(x) = \sum_{i=1}^r (v_i^T x + c_i)^+ - \sum_{i=1}^s (w_i^T x + d_i)^+ \quad (39)$$

satisfies $\sup_{x \in C} |h(x) - \hat{h}(x)| < \varepsilon$.

Reminder

The notation $(a)^+ = \max(a, 0) = \text{ReLU}(a)$. Thus the function $\hat{h}$ is a difference of two ReLU networks *with one hidden layer*. This result is a consequence of the Stone-Weierstrass approximation theorem applied to one-layer networks, or more directly of the universal approximation theorem of Cybenko (1989) / Hornik (1991) for networks with non-polynomial activation. For ReLU networks, see for example Leshno *et al.* (1993) or Barron (1993).

Lemma 9.6 (Reduction to a depth- I network). *Let $I \geq 2$ and $\hat{h}$ be of the form (39). Then for suitable integers $q_1, \dots, q_{I-1}$, the function $f^\theta = \mathbb{1}_{[0, +\infty)} \circ \hat{h}$ can be written as a network of the form (11)-(9) of depth I .*

Proof. Any function of the form (39) can be expressed using two ReLU layers with an affine output layer. One may always add identity layers (of the form $\text{ReLU}(x) - \text{ReLU}(-x) = x$) to reach the desired depth I , without changing the computed function. $\square$

A.8 J-sweep results

Table 4 provides detailed numerical results corresponding to the J-sweep experiment discussed in Section for the Bermudan max-call example.

As the nested sample size J increases, the gap between the upper and lower bounds decreases, while the computational time increases.

This result illustrates the trade-off between the tightness of the dual bound and the associated computational cost.

Table 4: J-sweep results for the Bermudan max-call example

J	Lower	Upper	Gap	Estimate	Time (s)
64	23.019	27.628	4.609	25.323	2.87
128	23.019	27.287	4.268	25.153	4.01
256	23.019	27.032	4.013	25.025	6.98
512	23.019	26.939	3.920	24.979	12.83
1024	23.019	26.882	3.863	24.950	23.99

A.9 Computational performance

Table 5 reports computational statistics collected during the training of neural network policies for the Bermudan max-call experiments. We used an NVIDIA Tesla T4 GPU.

Table 5: Computational performance statistics for Bermudan max-call experiments

d	S_0	Train time (s)	RSS (GB)	GPU peak (GB)
2	90	12.40	1.50	0.054
2	100	5.73	1.50	0.054
3	90	5.42	1.50	0.058
3	100	5.62	1.50	0.058
5	90	6.14	1.50	0.065
5	100	6.14	1.50	0.065

The reported values include wall-clock training time, CPU memory usage (RSS), and peak GPU memory usage. These statistics are obtained from the training logs of representative configurations.

These statistics directly address the project requirement of reporting wall-clock training time and memory usage.

References

- [1] Sebastian Becker, Patrick Cheridito, and Arnulf Jentzen. *Deep Optimal Stopping*. Journal of Machine Learning Research, 2019.
- [2] Paul Glasserman. *Monte Carlo Methods in Financial Engineering*. Springer, 2003.
- [3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [4] Andrew R. Barron. *Universal Approximation Bounds for Superpositions of a Sigmoidal Function*. IEEE Transactions on Information Theory, 1993.
- [5] Moshe Leshno and Shimon Schocken. *Multilayer Feedforward Networks with Non-Polynomial Activation Functions Can Approximate Any Function*. New York University, 1991.
- [6] Thi Bao Tram Ngo. Lecture notes, slides and code on deep learning. ENSIIE, 2025.
- [7] Karen Bozarian. Lecture notes and code on machine learning. ENSIIE, 2026.